《Smalltalk-80 应用编程：如何使用模型—视图—控制器（MVC）》中文译本

英文原题：Applications Programming in Smalltalk-80: How to Use Model-View-Controller (MVC)

作者：Steve Burbeck, Ph.D.

原文版权：© 1987、1992 S. Burbeck（作者允许出于教长或非商业目的复制）。

译文说明：这是保留历史版本语境的中文翻译。类名、消息名、Smalltalk 源代码与图示标识保留英文，便于核对原文；中文译文后附原 PDF 的全部 11 页。

作者说明

本文最初描述的是 Smalltalk-80 v2.0 中的 MVC 框架。1992 年修订时，加入了 Smalltalk-80 v2.5 中发生的变化。ParcPlace 后来在 4.x 版本中对相关机制进行了大量修改，这些修改没有反映在本文中。

`Smalltalk-80` 是 ParcPlace Systems, Inc. 的商标。

引言（Introduction）

Xerox Palo Alto研究中心（Xerox PARC）对编程艺术的一项贡献，是 Smalltalk-80 那种支持多个窗口、高度交互的用户界面。后来 Apple Lisa 和 Macintosh 的开发者借鉴了这种界面，而 Macintosh 众多模仿者又借鉴了它。在这类界面中，输入主要通过Mouse完成，输出则按需要混合使用图形和文字元素。

Smalltalk-80 用户界面背后的核心概念是模型—视图—控制器（Model–View–Controller，MVC）范式。它优美而简单，但与传统应用程序的做法相当不同。正因为这种新的做法性，它需要一些解释，而已出版的 Smalltalk-80 参考资料并没有很容易找到的相关说明。

如果你运行 `Pen` 类中的绘图示例，很可能会疑惑：为什么这个“应用”直接在屏幕上作画，而不是像你熟悉的 Browser、Workspace 或 Transcript 那样在一个窗口中显示？显然，你希望自己编写的应用也能像 Workspace 那样容易地与其他内容共用屏幕空间，而不是直接盖住整个屏幕。

区别究竟在哪里？简单地说：行为不规范的应用没有遵循 MVC 范式；那些熟悉的、行为合适的应用则遵循了它。

本文旨在向 Smalltalk-80 新程序员提供在自己程序中开始使用 MVC 技术所必需的信息。这里将介绍 MVC 的机制。理解这份入门材料以后，你就可以自己继续探索。

你需要进一步研究 Workspace、Browser、文件列表等熟悉的视图和控制器是如何建立的，以补充这里提供的信息。要尽早浏览代码，也要经常浏览代码。别忘了，这是 Smalltalk-80：我们鼓励你复制已有实现。你可以从一个与你想制作的窗口相似的窗口复制起步，再修改它。不用怕。许多程序员为 Smalltalk-80 V2.5 映像作出的贡献，都是可以借鉴的基础。从实际意义上说，那是他们留给你的规物。

基本概念（Basic Concepts）

在 MVC 范式中，用户输入、外部世界的建模，以及向用户提供视觉反馈，这三件事被明确分开，分别由三种专门负责自己任务的对象处理。

视图（View）管理分配给应用的位图显示区域中的图形和/或文字输出。控制器（Controller）解释用户的Mouse与键盘输入，并根据需要命令模型和/或视图发生变化。最后，模型（Model）管理应用问题领域中的行为与数据，响应获取自身状态信息的请求（通常来自视图），也响应要求改变状态的指令（通常来自控制器）。

将这三项任务在形式上分开，是一个重要思想，尤其适合 Smalltalk-80。在 Smalltalk-80 中，可以把基本行为封装进抽象对象 `View`、`Controller`、`Model` 和 `Object`，再通过继承、添加和必要的修改，构建灵活而强大的系统。

要有效使用 MVC，你必须理解 MVC 三元组内部的职责分工，也必须理解三个部分如何相互通信，以及如何与其他活动视图和控制器通信。多个应用共用一个Mouse、一个键盘和一块显示屏，因此必须进行通信与合作。为了充分利用 MVC，你还要了解 `View` 和 `Controller` 已有的子类，它们能作为应用的现成起点。

在 Smalltalk-80 中，输入和输出的方式在很大程度上是标准化的。视图必须管理自己的屏幕区域，并在其中显示文字或图形。控制器必须合作，确保由正确的控制器来解释键盘与Mouse输入（通常根据光标位于哪个视图内来决定）。

由于大多数应用的输入输出行为都遵循类似模式，因此可以从通用的 `View` 和 `Controller` 类继承大量行为。这两个类及其子类提供了非常丰富的功能，以至于应用通常只需要添加少量消息共议，就能完成命令输入与交互式输出。

与之相反，模型不能被强制标准化。如果限制哪些对象可以作为模型，就会限制 MVC 范式能够支持的应用范围。因此，任何对象都可以充当模型。

例如，一个浮点数可以是空速仪表视图的模型，而该仪表又可能是更复杂的飞行模拟器仪表盘视图的子视图。一个 `String` 完全可以成为文本Editor应用的模型（虽然通常使用稍复杂一点的 `StringHolder`）。

因为任何对象都可以充当模型的角色，所以模型参与 MVC 所需的基本行为，必须从 `Object` 类继承；`Object` 是所有可能充当模型的类的共同上层类。

MVC 三元组内部的通信（Communication Within the MVC Triad）

一个应用要与用户保持连一的交互，MVC 三元组中的模型、视图和控制器就必须彼此通信。视图和关联控制器之间的通信比较直接，因为 `View` 和 `Controller` 本来就是为合作而设计的。模型的通信方式则更加微妙。

被动模型（The Passive Model）

在最简单的情况下，模型完全不必为参与 MVC 三元组作任何特殊准备。一个简单的所见即所得（WYSIWYG）文本Editor就是很好的例子。

这种Editor的核心特征，是你应当始终看到文本打印到纸上时会呈现的样子。因此，每当文本发生变化，视图都必须得知，以便更新显示。

然而，模型（这里假设是 `String` 实例）并不一定要负责把变化通知视图，因为这些变化只会根据用户的请求发生。控制器负责解释用户请求，因此完全可以承担通知视图的责任。

控制器可以简单地告诉视图“某些内容发生了变化”，随后视图再向字符组模型请求当前状态；或者，控制器也可以直接告诉视图具体发生了什么变化。

无论哪种方式，字符组模型都是一个完全被动的数据持有者。它按照控制器的要求添加、移除或替换子组，并按照视图的请求返回相应的子组。模型完全不知道视图或控制器的存在，也不知道自己参与了一个 MVC 三元组。

这种隔离并不是因为模型太简单，而是因为：模型只会在三元组中其他成员要求它改变时才发生变化。

模型与三元组的连接（The Model's Link to the Triad）

但是，并非所有模型都能如此被动。假设数据对象——上例中的字符组——可能因为视图或控制器之外的其他对象向它发送消息而改变。例如，`SystemTranscript` 就可能不断向字符组末尾追加子组。

在这种情况下，依赖模型状态的对象（即它的视图）必须收到模型发生变化的通知。只有模型自己才能追踪其状态的全部变化，因此模型必须拥有某种通往视图的通信方式。

为满足这一需求，`Object` 提供了一个全局机制，用来追踪模型与视图等对象之间的依赖关系。这个机制使用一个名为 `DependentFields` 的 `IdentityDictionary`（`Object` 的类变量），记录已有的全部依赖关系。字典的键是那些加入过依赖对象的对象，键对应的值则是依赖于该对象的对象列表。

除此之外，`Model` 类提供了效率更高的依赖对象管理机制。创建准备在 MVC 三元组中充当主动模型的新类时，应当让它们继承 `Model`。

这一继承体系中的模型把依赖对象保存在实例变量 `dependents` 中，其值可以是 `nil`、单个依赖对象，或者一个 `DependentsCollection` 实例。视图依靠这些机制获知模型的变化。当一个新视图被指定模型时，

它会把自己加入为模型的依赖对象；视图释放时，则移除这种依赖关系。

提供这种间接依赖通信方式的方法，位于 `Object` 类的 `updating` 消息共议中。打开 Browser 看看这些方法。

`changed` 消息会向一个对象的所有依赖对象通知：这个对象发生了变化。收到 `changed` 的对象会向每个依赖对象发送 `update: self`。因此，模型只要执行 `self changed`，就可以通知所有依赖于自己的视图。

视图（以及加入为模型依赖对象的其他对象）收到的 `update:` 消息，以模型对象为参数。注意，还有一个 `changed:with:` 消息，可以向依赖对象转交额外参数。

从 `Object` 继承的默认 `update:` 方法什么也不做；但多数视图会实现收到 `update:` 后重新绘制自身的消息共议。

之所以选择 `changed/update:` 机制作为模型变化通知视图的方式，是因为它对模型结构施加的约束最少。

为了理解 MVC 中如何使用它，可以打开两个 Browser：一个查看 `changed` 的所有发送者（`Smalltalk browseAllCallsOn: #changed`），另一个查看 `update:` 的所有实现者（`Smalltalk browseAllImplementorsOf: #update`）。

你会发现，`update:` 的实现者几乎都是各种 `View`，它们的行为是更新显示。你自己的视图也会做类似的事情。`changed` 和 `changed:` 的发送者，则往往位于那些修改了对象某种重要属性、且这种修改需要反映到视图中的方法里。你以后对 `changed` 的使用方式也会与它们相近。

一个对象可以同时作为多个 MVC 三元组的模型。设想一座建筑的建筑学模型，暂时忽略模型内部的结构。重要的是，它可以拥有平面图视图、建筑外部透视图视图，以及展示建筑对外散热的视图（用于估算能源效率）。每个视图都有与自己配合的控制器。

模型变化时，可以通知全部依赖视图。如果只有部分视图应当响应某种变化，模型可以通过 `changed:` 传入参数，说明发生了哪一种变化，让只有真正感兴趣的视图作出响应。每个消息接收者都可以检查参数，决定应该采取什么行动。

视图—控制器之间的连接（The View–Controller Link）

与可以易散连接多个 MVC 三元组的模型不同，每个视图都关联着一个唯一的控制器，反过来也一样。双方的实例变量保存着这种紧密联系：视图的 `controller` 变量指向控制器，控制器的 `view` 变量指向视图。

由于两者都需要与模型通信，它们也各自有一个名为 `model` 的实例变量，指向模型对象。因此，模型只能通过类似 `self changed:` 的机制发出一般性的通知，而视图和控制器却能彼此直接发送消息，也能直接向模型发送消息。

在给定 MVC 三元组内部，建立这种通信关系是视图的责任。视图收到 `model:controller:` 消息后，会把自己加入为模型的依赖对象，让自己的 `controller` 实例变量指向指定控制器，并向控制器发送 `view:`

self，让控制器设置自己的 view 实例变量。

视图还负责解除这些连接。执行 View release 时，视图会先从模型的依赖对象列表中移除自己，再向控制器发送 release，最后对各个子视图发送 release。

视图（Views）

视图与子视图层级（The View/SubView Hierarchy）

视图被设计为可以嵌套。实际上，多数窗口至少包含两层视图，一层位于另一层内部。

最外层的视图称为 topView，通常是 StandardSystemView 或其子类的实例。StandardSystemView 管理窗口熟悉的标题文字标记；与它关联的控制器是 StandardSystemController 的实例，负责顶层窗口的移动、调整边框、折叠和关闭等操作。

topView 内部包含一个或多个子视图及其控制器，子视图管理自己区域内可用的交互功能。例如，常见 Workspace 以 StandardSystemView 为顶层视图，以 StringHolderView 为唯一子视图。

子视图还可以继续拥有子视图，虽然大多数应用并不需要这么做。子视图与上层视图的关系保存在从 View 继承的实例变量中：每个视图的 superView 指向包含它的视图，subViews 则是存放全部子视图的 OrderedCollection。

因此，每个窗口的 topView 都位于一个视图层级的顶部；沿着 superView/subViews 变量就可以追踪整个层级。

但要注意，有些视图类，例如 BinaryChoiceView 和 FillInTheBlankView，没有标题文字标记，也不能改变大小或移动。它们不使用 StandardSystemView 作为顶层视图，而是使用普通 View。

接下来看看建立并启动一个 MVC 三元组的例子。这是打开 methodListBrowser 的代码简化版。你在系统 Browser 的方法列表子视图中选择 implementors 或 senders 选项表项时，就会看到这种 Browser。

这种 Browser 的上部子视图显示方法列表；选择其中一个方法时，其代码显示在下部子视图。下面的代码附有行号，便于说明：

```
openListBrowserOn: aCollection label: labelString initialSelection: sel
    "Create and schedule a Method List browser for the methods in aCollection."
    | topView aBrowser |

1.  aBrowser := MethodListBrowser new on: aCollection.
2.  topView := BrowserView new.
3.  topView model: aBrowser; controller: StandardSystemController new;
4.          label: labelString asString; minimumSize: 300@100.
5.  topView addSubView:
6.      (SelectionInListView on: aBrowser printItems: false oneItem: false
7.          aspect: #methodName change: #methodName: list: #methodList
8.          menu: #methodMenu initialSelection: #methodName)
```

```
9.      in: (0@0 extent: 1.0@0.25) borderWidth: 1.
10. topView addSubView:
11.     (CodeView on: aBrowser aspect: #text change: #acceptText:from:
12.         menu: #textMenu initialSelection: sel)
13.     in: (0@0.25 extent: 1@0.75) borderWidth: 1.
14. topView controller open
```

现在逐行研究这段代码。第 [1] 行建立模型，第 [2] 行建立顶层视图。一般情况下，它会是 `StandardSystemView`；这里用的是其子类 `BrowserView`。

第 [3] 行指定模型和控制器。注意：如果没有显式提供控制器，当第一次请求视图的控制器时，视图的 `defaultController` 方法会提供控制器。许多应用会通过这个默认方法间接指定控制器，而不是在打开视图时显式设置。

第 [4] 行设置顶层视图的标题和最小尺寸。第 [5–9] 行布装上部的 `SelectionInListView` 子视图。第 [10–13] 行布装下部的 `CodeView`。这两类视图都称作可插接视图（Pluggable View），后面还会详细介绍。

请特别注意第 [9] 和 [13] 行，它们指定子视图如何放置在顶层视图占用的矩形内。`View` 的 `subView inserting` 消息共议中还有其他放置方法，例如 `addSubView:below:` 和 `insertSubView:above:`。

这里使用一个规范化的 `1.0@1.0` 矩形来指定相对位置。因此，代码不必依赖顶层窗口最终的尺寸或形状。

上部视图放在左上角 `0@0`，占据全部宽度，但只占顶层视图高度的前 25%，即 `extent: 1@0.25`。下部子视图从 `0@0.25` 开始，占据剩余区域，即 `1@0.75`。两者的边框宽度都是 1 像素。

最后，第 [14] 行打开控制器，使窗口开始调整边框的流程。此时光标变成窗口左上角的形状，让用户在屏幕上指定窗口大小。你自己的 MVC 应用通常也会以类似方式打开。

显示视图（Displaying Views）

你自己的视图可能需要专门的显示消息共议。该共议既用于第一次显示，也用于模型通知变化后的重新显示，还可能用于控制器主动要求的重新显示。

具体来说，`View` 的 `update:` 方法发送 `self display`。而 `View display` 依次发送：

```
self displayBorder.
self displayView.
self displaySubviews.
```

如果视图除了继承来的显示行为外，还需要任何特殊行为，就应把它放进这三个方法之一。可以查看 `displayView` 的不同实现者，了解各种绘制技术。你会发现，有些显示方法使用显示变换（Display Transform）。

显示变换是 `WindowingTransformation` 的实例。它处理缩放和平移，用来连接窗口（Window）与视口（Viewport）。这里的“窗口”是变换的输入：它是某个抽象显示空间中的矩形，可以采用最适合应用的任意坐标系。

视口则可以理解为显示屏上的某个具体矩形区域；抽象窗口应当被映射到那里。`WindowingTransformation` 计算缩放和平移系数，并将其应用到点、矩形等可显示对象，使 `aWindow` 经过变换后对应 `aViewport`。

不过，这种变换只是把一组坐标缩放和平移到另一组坐标，并不必然与显示屏有关；它也可以用于其他用途。

多个 `WindowingTransformation` 可以复合，逆变换也可以被调用。视图使用这些变换管理子视图的位置。如果需要直接在视图中绘图，你也可以使用它们。

`View displayTransform: anObject` 把当前视图的显示变换作用于 `anObject`，返回经过缩放和平移的对象。通常把它用于矩形、点等坐标定义在视图局部空间中的对象，以获得屏幕显示坐标中的结果。

`View displayTransformation` 返回一个 `WindowingTransformation`，这个结果由视图自身的局部变换以及沿着全部 `superView` 链上的局部变换复合而成。

控制器的 `redButtonActivity` 会使用 `View inverseDisplayTransformation: aPoint`，将 `aPoint`（例如 `Sensor cursorPoint`）从屏幕坐标转换为视图的窗口坐标。

关于已有视图的说明（Notes on Existing Views）

你最初编写的应用视图，应该从已有视图开始。本节末尾列出了带简要说明的现有视图。有些视图，如 Browser、Inspector、Debugger，是可以用作参考的完整应用；另一些是通用视图，可以原样用作应用中的子视图。你无疑还会希望继承某些子类，提供更专门的行为。

要了解某个视图如何使用，一个很有效的方法是浏览创建该视图的那些方法。例如，执行：

```
Smalltalk browseAllCallsOn: (Smalltalk associationAt: #SwitchView)
```

这会打开一个方法 Browser，显示全部向 `SwitchView` 类发送消息的方法。其中包含其他视图的实例创建方法，它们把 `SwitchView` 当作子视图使用。你可以以这些方法为范例。

四种现有通用视图特别灵活：`BooleanView`、`SelectionInListView`、`TextView` 和 `CodeView`。它们被称为可插接视图（Pluggable Views）。

它们增加灵活性，是为了减少某些子类的数量：这些子类往往只有获取模型数据、提供不同的 `yellowButtonMenu` 等常见任务所采用的消息选择子不同。

可插接视图及其控制器，通过调用实例创建时传入的“适配器”（Adaptor）消息选择子，来完成上述任务。前面 `openListBrowserOn:` 方法里使用的 `SelectionInListView` 和 `CodeView` 就是例子。注意，传给这两种视图创建方法的参数是选择子名称。每个可插接视图的类注释会说明需要传入哪些选择子。

已有视图层级（The Existing View Hierarchy）

下面保留原文所列的主要继承层级，并补充各类用途：

- `View`：用作 `BinaryChoiceView`、`FillInTheBlankView` 的非标准顶层视图。
- `BinaryChoiceView`：带可定/否定图示的提示器。
- `SwitchView`：在 `BitEditor` 和 `FormEditor` 中充当工具按键。
- `BooleanView`：可插接视图，用于 Browser 中实例侧与类侧的切换。
- `DisplayTextView`：用于是/否提示器上部子视图中的消息文字。
- `TextView`：可插接；在标准 V2.5 映像中没有直接使用。
- `CodeView`：可插接；用于显示代码的 Browser 子视图。
- `OnlyWhenSelectedCodeView`：用于 `FileList` 下部子视图。
- `StringHolderView`：用于 Workspace。
- `FillInTheBlankView`：要求输入文字回答的常见提示器。
- `ProjectView`：项目的说明视图。
- `TextCollectorView`：用于 Transcript。
- `FormMenuView`：用于 `BitEditor` 和 `FormEditor` 的按键。
- `FormView`：用于 `BitEditor` 和作为灰色屏幕背景的 `InfiniteForm`。
- `FormHolderView`：用于 `BitEditor`、`FormEditor` 中正在修改的图形。
- `ListView`：在标准 V2.5 映像中没有直接使用。
- `ChangeListView`：一个完整应用。
- `SelectionInListView`：可插接；用于 Browser 的列表子视图。
- `StandardSystemView`：提供顶层视图的功能。
- `BrowserView`：完整应用。
- `InspectorView`：完整应用。
- `NotifierView`：错误通知器，例如提示“对象不理解某个消息”。

控制器（Controllers）

Smalltalk-80 给人的感觉是：控制权掌握在Mouse之中。随着Mouse移动和点击，屏幕上的对象就像交响快团听从指挥一样行动。

但实际上，并不存在一个独断的中心控制者。系统依靠各个活动视图所关联控制器之间的合作，维持单一的控制流。在任何一个时刻，真正拥有控制权的只能是一个控制器。因此，关键就是确保取得控制权的是正确的那个控制器。

控制器之间的通信（Communication Between Controllers）

实现这一点的主要组织原则是：每个项目中的活动控制器组成一个层级树。

树根是全局变量 `ScheduledControllers`，它是一个与活动项目关联的 `ControlManager`。从它向下分支的是每个活动窗口的顶层控制器，另外还有一个控制器负责灰色屏幕背景上可用的主系统 `yellowButtonMenu`。

由于每个视图都关联唯一的控制器，视图/子视图树会在每个 `topView` 内形成一个对应的控制器树。每个顶层控制器的进一步分支，就沿着这个树展开。控制权沿着树的分支在控制器之间转交。

简单说，控制流程依靠一组遵从规则的控制器共同完成。每个控制器都很有规矩：除非光标位于自己的视图内，否则它不会要求控制权。即便取得控制权，也会尝试把控制权让给某个子视图的控制器。

顶层 `ControlManager` 会问问所有活动顶层视图的控制器，看看人需要控制权。只有视图包含光标的控制器回答“需要”，并取得控制权。随后它又问问各个子视图的控制器；同样由包含光标的那个取得控制权。

这个过程会找到包含光标的最内层嵌套视图。一般情况下，只要光标一直止留在这个视图内，它的控制器就会保持控制权。（附录 A 对这一控制流程作了更详细的说明。）

在这套机制中，控制管理依赖所有活动视图和控制器的精细配合。视图需要轮问其子视图的控制器；控制器则问问自己的视图是否包含光标。

因此，修改视图或控制器、引入不符合标准的控制流程，既不常见，也可能有问题。第一次布装新的应用控制器时，请务必记住这一点，因为无意中破坏控制流可能导致整个系统系统出错。小心的程序员会在尝试前保存一次系统快照。

控制器如此重要，意味着不可能存在完全没有控制器的模型—视图对。否则，控制流会在缺失控制器的位置中断。

然而，有些情况下，我们希望一组子视图统一由容器的控制器控制，而不是分别由它们自己的控制器控制。为此，系统提供了 `NoController` 类。它专门被设计成始终拒绝取得控制权的控制器。

随着光标在 Browser 的子视图之间移动，各个滚动条时隐时现，这是控制流最直观的效果。当光标跨越子视图边界时，刚离开的子视图的滚动条消失，而进入的子视图的滚动条出现。

对于继承 `ScrollController` 的控制器，这由 `controlInitialize` 与 `controlTerminate` 方法完成。

光标离开某个子视图时，`viewHasCursor` 返回 `false`，控制器执行 `controlTerminate`，把先前被滚动条盖住的区域重新显示出来。随后系统重新遍历视图/控制器树的适当部分，寻找现在应该取得控制权的控制器。

如果新视图需要滚动条，新控制器的 `controlInitialize` 先保存即将被滚动条盖住的显示区域，然后把滚动条画出来。

进入和离开控制流（Entering and Leaving the Flow of Control）

请记住，这场“规矩的共同流程”一直在进行。那么，新建立的 MVC 三元组怎样加入，又怎样在完成工作后退出?

首先，顶层视图的控制器负责加入这个过程。之后，它把控制权转交给子视图的控制器——在典型应用里，真正完成大部分工作的正是这些子控制器。

全部顶层控制器都必须继承 `StandardSystemController`，这个类就是专为顶层视图的控制器而设计的。

向 `StandardSystemController` 发送 `open` 消息，会使新的 MVC 成为控制器树的一个顶层分支。`open` 必须是建立新 MVC 的方法中的最后一条消息，因为该消息不会正常返回控制权；它后面的代码不会执行。

关闭窗口时，`StandardSystemController` 的 `controlTerminate` 方法负责把该窗口从调度中移除。

MouseMenuController

多数应用使用Mouse来指向对象和选择选项表项。因此，大多数控制器都位于 `MouseMenuController` 下方的继承层级中。这个类提供基本的实例变量和方法。

相关的实例变量包括 `{red, yellow, blue}ButtonMenu` 和 `{red, yellow, blue}ButtonMessages`，分别存储选项表与对应的消息。重要的方法有 `redButtonActivity`、`yellowButtonActivity` 和 `blueButtonActivity`。

顾名思义，`Controller controlLoop` 是主控制循环。每次循环都会发送 `self controlActivity`。`MouseMenuController` 盖住此方法以检查每个Mouse键；例如，如果红色键被按下，且当前视图包含光标，就发送 `self redButtonActivity`。

相应的 `xxxButtonActivity` 会检查 `xxxButtonMenu` 是否非 `nil`。如果存在选项表，就在用户选择某项后执行类似下面的消息：

```
self menuMessageReceiver perform: (redButtonMessages at: index)
```

注意：menuMessageReceiver 通常返回 self，即控制器本身。因此，选项表消息共议通常定义在控制器内。

全部顶层控制器都是 StandardSystemController 或其子类的实例。它继承自 MouseMenuController，但专门负责窗口控制器层级的顶部。它管理熟悉的 blueButtonMenu，其中的窗口定位、关闭、折叠等操作作用于 topView。

子视图控制器不应该继承 StandardSystemController，而应该独立地继承 MouseMenuController。不过，蓝色按键的选项表功能仍应由顶层控制器处理。

为了保证这一点，子视图控制器可以通过如下 blueButtonActivity，把蓝色按键操作转交顶层：

```
blueButtonActivity
    view topView controller blueButtonActivity.
```

这种做法很直观。浏览控制器代码的人能立即看出，蓝色按键由 topView 的控制器处理。

另一种更隐盖的方法，是让子视图控制器在蓝色按键按下时拒绝控制权。可以在 isControlActive 中这样实现：

```
isControlActive
    ^super isControlActive & sensor blueButtonPressed not.
```

在典型情况下，控制器拥有自己的 yellowButtonMenu，也可能把红色按键用于指向或选择列表项等功能。选项表通常可以像下面这样建立：

```
PopUpMenu labels:
    'foo baz
     over there
     file out
     new gadget'
    lines: #(1 3).
```

这会生成包含上述选项的选项表，并在第一个和第三个选项表项后显示分隔线。

随后布装一个与选项表项一一对应的消息列表，例如 #(fooBaz overThere fileOut newGadget)。选项表和消息列表最终必须存入 yellowButtonMenu 与 yellowButtonMessages 实例变量。

可以在运行时随时建立这些内容，但更典型的做法是在类侧 initialize 中构建选项表和消息，再在实例侧 initialize 中布装。可以参考 ChangeListController 的类侧 initialize。

最后，还需要实现这些选项表消息的方法。按照惯例，它们放在控制器的 menu messages 消息共议中。

如果希望红色按键执行选项表选择以外的操作，可以盖住控制器的 redButtonActivity。在新方法中，还可以检查 Sensor leftShiftDown 等其他条件，让操作更灵活。FormEditor、ListController 和 SelectionInListController 都有不以选项表为目的的 redButtonActivity 实例可供参考。

虽然选项表也使用显示屏，但并不是由 MVC 三元组处理的。选项表会自行管理屏幕显示和控制，包括保存选项表盖住区域原来的内容，并在选项表结束时恢复它们。除了上面的 `PopUpMenu`，你还应研究 `ActionMenu`，它对可插接视图特别有用。

ParagraphEditor

在 `Controller` 的继承层级中，所有接受键盘文本输入的控制器都位于 `ParagraphEditor` 之下。

`ParagraphEditor` 出现的时间早于 MVC 范式的完整发展。它同时处理文本输入和显示，因此在某些方面像是视图与控制器的混合体。使用它或其子类作为控制器的视图，会把通常的显示职责反过来：它们通过向控制器发送 `display` 消息来显示文本。

`ParagraphEditor` 承担了许多不同职责，所以是一个复杂对象。它负责特殊按键（例如把 Ctrl+T 映射到 `ifTrue:`）、文本选区、字体和字号的选择，以及文本排版（按照视图宽度进行比例字符间距和换行处理）。

由于它位于文本处理控制器层级的顶部，继承它便能方便地获得这些强大功能。但文本处理类的复杂性也带来相当大的性能开销。最明显的表现，是打下一个字符到字符显示在屏幕上之间，那种令人不快的延迟。

承担大部分文本处理工作的三个类是 `ParagraphEditor`、`Paragraph` 和 `CharacterScanner`。一些 Smalltalk-80 程序员为这三个类分别建立了简化版本，舍弃大部分耗时功能，使专门用途的应用获得比标准类快几倍的文本处理速度。

ScreenController

灰色屏幕背景及其可用的 `yellowButtonMenu` 并不是 MVC 范式的例外：它们也由一个 MVC 三元组管理。

模型是一个灰色的 `InfiniteForm`，视图是 `FormView` 的实例，而控制器是 `ScreenController` 类的唯一实例。

如果希望在主屏幕选项表中添加一项（例如 `printScreen`），可以修改 `ScreenController` 类侧的 `initialize` 方法，并把相应方法加入 `menu messages` 共议。别忘了执行类侧 `initialize` 方法末尾的注释指示，以布装新选项表。

你也可以查看这个消息共议中的其他方法，了解 Browser、文件列表或 Workspace 是怎样打开的。

MVC 检查器（The MVC Inspector）

MVC 三元组如此重要，Smalltalk-80 因而提供了一种专用 Inspector——MVC Inspector，可以同时检查三个对象。

开始构建自己的应用时，你很可能会经常使用它：既可以研究其他 MVC 三元组的工作方式，又可以在自己的应用失效时帮助调试。

只要向任意 View 发送 inspect 消息，就能打开一个 MVC Inspector。

使用 MVC Inspector 的最简单方式，是导入 Smalltalk-AT 发行包附带的 BLUEINSP.ST 扩展文件（Goodie）。注意，并非所有 Smalltalk-80 发行包都包含它。

这个扩展会为所有顶层视图添加一个蓝色按键选项表项 inspect。选择该项，就会打开一个 MVC Inspector，一起检查视图、模型和控制器。

只要对 MVC 三元组有一定了解，这个工具就能让你深入任何感兴趣的窗口内部。在拥有许多子视图的复杂窗口（如 Browser）中，还可以沿着 subViews 链不断向下，对选中的子视图继续打开新的 MVC Inspector。

下面是一个实践：对 System Transcript 窗口打开 MVC Inspector。首先要确保屏幕上已经打开 SystemTranscript。

先执行 DependentsCollection allInstances inspect，打开对 DependentsCollection 所有实例的检查。找到具有两个依赖对象的那一项：一个是 StandardSystemView，另一个是 TextCollectorView。

选中 TextCollectorView 并执行 inspect。现在，你会得到一个 MVC Inspector，可同时查看 TextCollector、它的视图和控制器。

例如，在上方的模型部分选中 contents 变量；右边窗口中会出现与 Transcript 窗口中相同的文本。在底部控制器部分查看并检查按键选项表和对应消息，你会看到 Transcript 中熟悉的那些选项表项。

再到中间的视图部分选择 superView；它是一个 StandardSystemView。选中它并执行 inspect，就会打开另一个 MVC Inspector，查看 Transcript 窗口的顶层视图。

这两个 MVC Inspector 合起来，就可以访问整个 Transcript 应用的结构。注意，两个 MVC 三元组共用同一个模型——TextCollector。

旧版本说明： 在尚未引入 Model 类的 Smalltalk-80 版本中，开始方法不同。先执行：

```
(Object classPool at: #DependentsFields) inspect.
```

这会打开对存储全部对象依赖关系的 IdentityDictionary 的检查。找到键为 aTextCollector 的条目，也就是 Transcript 的模型。选中它并在 yellowButtonMenu 里选择 inspect，查看加入为 TextCollector 依赖对象的 OrderedCollection。其中会有一个 TextCollectorView；选中它并检查即可。

附录 A：控制流的细节（Details on the Flow of Control）

在所有顶层视图控制器之上，还有 ScheduledControllers——它是与当前项目关联的 ControlManager 实例。这个控制管理器决定哪个顶层视图控制器应当处于活动状态。

它使用 `searchForActiveController` 方法，后者向所有已调度控制器发送 `isControlWanted`。找到适当的顶层控制器后，就向它发送从 `Controller` 类继承的 `startUp`。

`startUp` 依次发送以下三条消息：

```
self controlInitialize.
self controlLoop.
self controlTerminate.
```

其中 `controlLoop` 通过下面的循环处理控制流：

```
[self isControlActive] whileTrue: [
    Processor yield.
    self controlActivity
]
```

而 `self controlActivity` 的基本行为，只是执行 `self controlToNextLevel`。在这里，控制流通过如下代码短暂转交给视图：

```
aView := view subViewWantingControl.
aView ~~ nil ifTrue: [aView controller startUp]
```

这段代码要求视图决定哪个子视图想取得控制权。如果存在，就把控制权交给该子视图的控制器。

`isControlWanted` 只是返回 `self viewHasCursor` 的结果，而后者又只是执行：

```
view containsPoint: sensor cursorPoint
```

因此，整个控制流判定过程的最底层问题，就是“哪个视图包含光标”。唯一的例外是 `NoController`，它盖住 `isControlWanted`，让该方法始终返回 `false`。

感谢 Charles Lloyd 将本文转换为 HTML。若发现错误，请联系 Ian。（原文结尾说明。）

译者说明（不属于原文）

- 本文讨论的是 Smalltalk-80 v2.0/v2.5 的具体机制，而不是所有现代 MVC 框架的统一规范。
- `Passive Model` 是作者用来区分通知责任的一个例子：如果状态变化全由控制器发起，控制器可以通知视图；若模型能被其他对象修改，就需要模型自己的变更通知机制。
- 本文明确区分 `Object` 的全局 `DependentFields` 机制与 `Model` 实例变量 `dependents` 所实现的依赖管理；不能把它们混为一种物理存储方式。
- 本文的控制器树和单一活动控制器调度，是当时 Smalltalk 窗口系统的具体实现，不应直接当成 Unity UGUI 或 UI Toolkit 的底层机制。

原始英文扫描页（供核对图表与源码）

以下保留原 PDF 的完整页面，包括图表、程序列表、代码注释与历史页码。前面的中文译文引用的图号均可在这些原页中找到。

Applications Programming in Smalltalk-80 (TM): How to use Model-View-Controller (MVC)

by
Steve Burbeck, Ph.D.

Author's note: This paper originally described the MVC framework as it existed in Smalltalk-80 v2.0. It was updated in 1992 to take into account the changes made for Smalltalk-80 v2.5. ParcPlace made extensive changes to the mechanisms for versions 4.x that are not reflected in this paper.

Copyright (c) 1987, 1992 by S. Burbeck
permission to copy for educational or non-commercial purposes is hereby granted

(TM)Smalltalk-80 is a trademark of ParcPlace Systems, Inc.

Introduction

One of the contributions of Xerox PARC to the art of programming is the multiwindowed highly interactive Smalltalk-80 interface. This type of interface has since been borrowed by the developers of the Apple Lisa and Macintosh and, in turn, by the Macintosh's many imitators. In such an interface, input is primarily by mouse and output is a mix of graphic and textual components as appropriate. The central concept behind the Smalltalk-80 user interface is the *Model-View-Controller* (MVC) paradigm. It is elegant and simple, but quite unlike the approach of traditional application programs. Because of its novelty, it requires some explanation -- explanation which is not readily available in published Smalltalk-80 references.

If you run the graphics example in class *Pen*, you might well wonder why this "application" draws directly on the screen rather than in a window like the browsers, workspaces, or transcripts with which you are familiar. Certainly you would wish your own applications to share space on the display with the easy aplomb of a workspace rather than simply overwrite the screen. Just what is the difference? Most simply put, ill behaved applications do not conform to the MVC paradigm, whereas the familiar well behaved applications do.

This paper is intended to provide the information essential for new Smalltalk-80 programmers to begin using MVC techniques in their own programs. Here we will introduce the mechanisms of MVC. Once you have digested this introduction you can strike out on your own. You will need to flesh out the information given here by looking at the way familiar kinds of views and controllers -- such as workspaces, browsers and file lists -- are set up. Browse early and often. Remember, this is Smalltalk-80. You are encouraged to copy. Start your own window by copying one that is similar to the one you want to create. Then modify it. Don't be shy. Feel free to stand on the shoulders of the many programmers who htridave contributed to the Smalltalk-80 V2.5 image. In a very real way, it is their gift to you.

Basic Concepts

In the MVC paradigm the user input, the modeling of the external world, and the visual feedback to the user are explicitly separated and handled by three types of object, each specialized for its task. The **view** manages the graphical and/or textual output to the portion of the bitmapped display that is allocated to its application. The **controller** interprets the mouse and keyboard inputs from the user, commanding the model and/or the view to change as appropriate. Finally, the **model** manages the behavior and data of the application domain, responds to requests for information about its state (usually from the view), and responds to instructions to change state (usually from the controller). The formal separation of these three tasks is an important notion that is particularly suited to Smalltalk-80 where the basic behavior can be embodied in abstract objects: *View*, *Controller, Model* and *Object*. The MVC behavior is then inherited, added to, and modified as necessary to provide a flexible and powerful system.

To use the MVC paradigm effectively you must understand the division of labor within the MVC triad. You also must understand how the three parts of the triad communicate with each other and with other active views and controllers; the sharing of a single mouse, keybord and display screen among several applications demands communication and cooperation. To make the best use of the MVC paradigm you need also to learn about the available subclasses of *View* and *Controller* which provide ready made starting points for your applications.

In Smalltalk-80, input and output are largely stylized. Views must manage screen real estate and display text or graphic forms within that real estate. Controllers must cooperate to ensure that the proper controller is interpreting keyboard and mouse input (usually according to which view contains the cursor). Because the input and output behavior of most applications is stylized, much of it is inherited from the generic classes -- *View* and *Controller*. These two classes, together with their subclasses, provide such a rich variety of behavior that your applications will usually require little added protocol to accomplish their command input and interactive output behavior. In contrast, the model cannot be stylized. Constraints on the type of objects allowed to function as models would limit the useful range of applications possible within the MVC paradigm. Necessarily, any object can be a model. A float number could be the model for an airspeed view which might be a subview of a more complex flight simulator instrument panel view. A *String* makes a perfectly usable model for an editor application (although a slightly more complex object called a *StringHolder* is usually used for such purposes). Because any object can play the role of model, the basic behavior required for models to participate in the MVC paradigm is inherited from class *Object* which is the class that is a superclass of all possible models.

Communication Within The MVC Triad

The model, the view and the controller involved in the MVC triad must communicate with each other if an application is to manage a coherent interaction with the user. Communication between a view and its associated controller is straightforward because *View* and *Controller* are specifically designed to work together. Models, on the other hand, communicate in a more subtle manner.

The Passive Model

In the simplest case, it is not necessary for a model to make any provision whatever for participation in an MVC triad. A simple WYSIWYG text editor is a good example. The central property of such an editor is that you should always see the text as it would appear on paper. So the view clearly must be informed of each change to the text so that it can update its display. Yet the model (which we will assume is an instance of *String*) need not take responsibility for communicating the changes to the view because these changes occur only by requests from the user. The controller can assume responsibility for notifying the view of any changes because it interprets the user's requests. It could simply notify the view that something has changed -- the view could then request the current state of the string from its

model -- or the controller could specify to the view what has changed. In either case, the string model is a completely passive holder of the string data manipulated by the view and the controller. It adds, removes, or replaces substrings upon demand from the controller and regurgitates appropriate substrings upon request from the view. The model is totally "unaware" of the existence of either the view or the controller and of its participation in an MVC triad. That isolation is not an artifact of the simplicity of the model, but of the fact that the model changes only at the behest of one of the other members of the triad.

The Model's Link to the Triad

But all models cannot be so passive. Suppose that the data object -- the string in the above example -- changes as a result of messages from objects other than its view or controller. For instance, substrings could be appended to the end of the string as is the case with the SystemTranscript. In that case the object which depends upon the model's state -- its view -- must be notified that the model has changed. Because only the model can track all changes to its state, the model must have some communication link to the view. To fill this need, a global mechanism in *Object* is provided to keep track of dependencies such as those between a model and its view. This mechanism uses an *IdentityDictionary* called *DependentFields* (a class variable of *Object*) which simply records all existing dependencies. The keys in this dictionary are all the objects that have registered dependencies; the value associated with each key is a list of the objects which depend upon the key. In addition to this general mechanism, the class *Model* provides a more efficient mechanism for managing dependents. When you create new classes that are intended to function as active models in an MVC triad, you should make them subclasses of *Model*. Models in this hierarchy retain their dependents in an instance variable (*dependents*) which holds either nil, a single dependent object, or an instance of *DependentsCollection*. Views rely on these dependence mechanisms to notify them of changes in the model. When a new view is given its model, it registers itself as a dependent of that model. When the view is released, it removes itself as a dependent.

The methods that provide the indirect dependents communication link are in the "updating" protocol of class *Object*. Open a browser and examine these methods. The message *"changed"* initiates an announcement to all dependents of an object that a change has occurred in that object. The receiver of the *changed* message sends the message *update: self* to each of it's dependents. Thus a model may notify any dependent views that it has changed by simply sending the message *self changed*. The view (and any other objects that are registered as dependents of the model) receives the message *update:* with the model object as the argument. [Note: There is also a *changed:with:* message that allows you to pass a parameter to the dependent.] The default method for the *update:* message, which is inherited from *Object*, is to do nothing. But most views have protocol to redisplay themselves upon receipt of an *update:* message. This *changed/update* mechanism was chosen as the communication channel through which views can be notified of changes within their model because it places the fewest constraints upon the structure of models.

To get an idea of how this changed/update: mechanism is used in MVC, open a browser on senders of the *changed* message (*Smalltalk browseAllCallsOn: #changed*) and another on implementers of the *update:* message (*Smalltalk browseAllImplementorsOf: #update*). Note that nearly all the implementors of *update*: are varieties of *View*, and that their behavior is to update the display. Your views will do something similar. The senders of *changed* and *changed:* are in methods where some property of the object is changed which is important to its view. Again, your use of the *changed* message will be much like these.

An object can act as a model for more than one MVC triad at a time. Consider an architectural model of a building. Let us ignore the structure of the model itself. The important point is that there could be a view of the floor plan, another external perspective view, and perhaps another view of external heat loss

(for estimating energy efficiency). Each view would have its cooperating controller. When the model is changed, all dependent views can be notified. If only a subset of these views should respond to a given change, the model can pass an argument which indicates to the dependents what sort of change has occurred so that only those interested need respond. This is done with the *changed:* message. Each receiver of this message can check the value of the argument to determine the appropriate response.

The View - Controller Link

Unlike the model, which may be loosely connected to multiple MVC triads, Each view is associated with a unique controller and vice versa. Instance variables in each maintain this tight coupling. A view's instance variable *controller* points at its controller, and a controller's instance variable *view* points at its associated view. And, because both must communicate with their model, each has an instance variable *model* which points to the model object. So, although the model is limited to sending *self changed:,* both the view and the controller can send messages directly to each other and to their model.

The View takes responsibility for establishing this intercommunication within a given MVC triad. When the View receives the message *model:controller:*, it registers itself as a dependent of the model, sets its controller instance variable to point to the controller, and sends the message *view: self* to the controller so that the controller can set its *view* instance variable. The View also takes responsibility for undoing these connections. View *release* causes it to remove itself as a dependent of the model, send the message *release* to the controller, and then send *release* to any subViews.

Views

The View/SubView Hierarchy

Views are designed to be nested. Most windows in fact involve at least two views, one nested inside the other. The outermost view, known as the topView is an instance of *StandardSystemView* or one of its subClasses. The *StandardSystemView* manages the familiar label tab of its window. Its associated controller, which is an instance of *StandardSystemController*, manages the familiar moving, framing, collapsing, and closing operations available for top level windows. Inside a topView are one or more subViews and their associated controllers which manage the control options available in those views. The familiar workspace for example has a *StandardSystemView* as a topView, and a *StringHolderView* as its single subView. A subView may, in turn, have additional subViews although this is not required in most applications. The subView/superView relationships are recorded in instance variables inherited from *View*. Each view has an instance variable, *superView*, which points to the view that contains it and another, *subViews*, which is an *OrderedCollection* of its subViews. Thus each window's topView is the top of a hierarchy of views traceable through the superView/subViews instance variables. Note however that some classes of view (e.g., *BinaryChoiceView,* and *FillInTheBlankView*) do not have label tabs, and are not resizable or moveable. These classes do not use the *StandardSystemView* for a topView; instead they use a plain *View* for a topView.

Let's look at an example which builds and launches an MVC triad. This example is a simplified version of the code which opens a methodListBrowser -- the browser you see when you choose the implementors or senders menu item in the method list subView of a system browser. The upper subView of this browser displays a list of methods. When one of these methods is selected, its code appears in the lower subView. Here is the code with lines numbered for easy reference.

```
openListBrowserOn: aCollection label: labelString initialSelection: sel
"Create and schedule a Method List browser for the methods in aCollection."
| topView aBrowser |
```

```
1.         aBrowser := MethodListBrowser new on: aCollection.
2.         topView := BrowserView new.
3.         topView model: aBrowser; controller: StandardSystemController new;
4.                               label: labelString asString; minimumSize: 300@100.
5.   topView addSubView:
6.                     (SelectionInListView on: aBrowser printItems: false oneItem: false
7.                     aspect: #methodName   change: #methodName:   list: #methodList
8.                     menu: #methodMenu   initialSelection: #methodName)
9.                     in: (0@0 extent: 1.0@0.25) borderWidth: 1.
10.   topView addSubView:
11.                    (CodeView on: aBrowser   aspect: #text   change: #acceptText:from:
12.                              menu: #textMenu   initialSelection: sel)
13.                              in: (0@0.25 extent: 1@0.75) borderWidth: 1.
14.   topView controller open
```

Now let's look at this code line by line. After creating the model [1], we create the topView [2]. Usually this will be a *StandardSystemView*, but here we use a *BrowserView*, which is a subClass of *StandardSystemView*. Line [3] specifies the model and controller. [Note: If the controller is not explicitly provided, the *defaultController* method of the view will provide the controller when the view's controller is first requested. Many applications specify the controller indirectly in this default method rather than explicitly providing the controller when the view is opened.] The next line provides the topView's label and minimum size [4]. Lines [5-9] install the upper subView, which is a *SelectionInListView*. The lower *CodeView* is installed by lines [10-13]. Both of these types of view are known as "pluggable views." These are discussed in more detail in a later section. Look closely at lines [9] and [13] which indicate the placement of the subViews within the rectangle occupied by the topView. There are a variety of ways to indicate to a view how it should place its subviews such as *addSubView:below:* and *insertSubView:above:*. They will be found in the subView inserting protocol of *View*. Here the placements are given relative to a canonical 1.0@1.0 rectangle. Your code need not depend upon the final size and shape of the topView window. The upper view is placed at the upper left corner (i.e., at 0@0) and allowed to occupy the full width but only the top 25% of the height of the topView (i.e. extent: 1@0.25). The lower subView is placed at 0@0.25 and allowed to occupy the remainder of the window (1@0.75). Each is given a borderWidth of 1 pixel. Finally, the controller is opened [14] which causes the window to initiate the framing process -- the cursor becomes the upper left corner cursor so that the user can frame the window. Your own MVC applications will usually be opened similarly.

Displaying Views

Your view may need its own display protocol. This protocol will be used both for the initial display of your view and for redisplay when the model signals a change (and possibly for redisplay instigated by the controller as well). Specifically, the *update:* method in *View* sends *self display*. *View display* in turn sends *self displayBorder. self displayView. self displaySubviews.* If your view requires any special display behavior other than what is inherited, it belongs in one of these three methods. You can browse implementors of *displayView* for examples of different sorts of display techniques. If you do, you will note that several display methods make use of display transforms.

Display transforms are instances of *WindowingTransformation*. They handle scaling and translating in order to connect windows and viewports. A window is the input to the transformation. It is a rectangle in an abstract display space with whatever arbitrary coordinate system you find most appropriate for your application. A viewport can be thought of as a specific rectangular region of the display screen to which the abstract window should be mapped. The class *WindowingTransformation* computes and applies scale and translation factors on displayable objects such as points and rectangles so that aWindow, when transformed, corresponds to aViewport. However the transformation simply scales and translates one set of coordinates to another hence there is no necessary connection to the display screen; the transformation could be used for other purposes.

*WindowingTransformation*s can be composed, and their inverses can be invoked. Views use transformations to manage subview placement. You too can use them if you need to draw directly in a view. *View displayTransform:* anObject applies the display transformation of the receiver to anObject and answers the resulting scaled, translated object. It is usually applied to Rectangles, Points, and other objects with coordinates defined in the View's local coordinate system to obtain a scaled and translated object in display screen coordinates. The *View displayTransformation* returns a *WindowingTransformation* that is the result of composing all local transformations in the receiver's superView chain with the receiver's own local transformation. *View inverseDisplayTransformation: aPoint* is used by controllers *redButtonActivity* to convert aPoint (e.g., Sensor cursorPoint) from screen coordinates to view's window coordinates.

Notes on Existing Views

Your first application views will begin with existing views, an annotated list of which appears at the end of this section. Some of these, such as browsers inspectors and debuggers are complete applications which you can use as examples. Others are general purpose views which you will use intact as subviews in your applications. Some you will no doubt want to refine by creating subclasses with more specialized behavior. Much can be learned about making use of a given view by simply browsing the view creation methods that make use of the view. For example, executing *Smalltalk browseAllCallsOn: (Smalltalk associationAt: #SwitchView)* will present you with a method browser on all methods which send messages to class SwitchView. Among these will be instance creation methods of other views which use the given view as a subview. You can use these as examples of how to do so yourself.

Four of the general purpose existing views -- *BooleanView*, *SelectionInListView*, *TextView* and *CodeView* -- are especially flexible. These are called "pluggable views." Their extra flexibility is designed to reduce the need for many subclasses which differ only in the method selectors used to do common tasks such as obtain data from the model or present a different *yellowButtonMenu*. Pluggable views and their associated controllers perform these tasks by invoking "adaptor" selectors passed to them at the time of instance creation. The *SelectionInListView* and *CodeView* used as subviews in the *openListBrowserOn:* method shown in an earlier section are examples. Note that the arguments passed to the creation methods of these views are selector names. The class comment of each pluggable view defines the selectors to be passed to that view.

The Existing View Hierarchy

```
View - used as nonstandard topView for BinaryChoiceView and FillInTneBlankView
          BinaryChoiceView - the thumbs up/down prompter
          SwitchView - used in BitEditor and FormEditor as tool buttons
                    BooleanView - [pluggable] used for browser instance/class switch
          DisplayTextView - used for message in the upper subview of a yes/no prompter
          TextView - [pluggable] not used in vanilla V2.5 image
                    CodeView - [pluggable] used for browser subview which shows code
                              OnlyWhenSelectedCodeView - used by FileList lower subview
          StringHolderView - used by workspaces
                    FillInTheBlankView - the familiar question prompter with a text answer
                    ProjectView - description view of a project
                    TextCollectorView - used by the Transcript
          FormMenuView - used by BitEditor and FormEditor for buttons
          FormView - used by BitEditor and the background screen gray InfiniteForm
                    FormHolderView - used by BitEditor and FormEditor for edited form
          ListView - not used in vanilla V2.5 image
                    ChangeListView - a complete application
                    SelectionInListView - [pluggable] used for browser list subviews
          StandardSystemView - provides topView functions
                    BrowserView - complete applications
                    InspectorView - complete application
                    NotifierView - error notifier, e.g., "Object does not understand"
```

Controllers

Smalltalk-80 presents the appearance that control resides in the mouse. As one moves and clicks the mouse, the objects on the Smalltalk-80 screen perform much as an orchestra obeying its conductor. But there is, in fact, no single autocratic power. A single thread of control is maintained by the cooperation of the controllers attached to the various active views. Only one controller can actually have control at any one time. So the trick is to make sure it is the proper one!

Communication Between Controllers

The primary organizing principle which makes this trick possible is that the active controllers for each project form a hierarchical tree. At the root of this tree is the global variable *ScheduledControllers*, which is a *ControlManager* attached to the active project. Branching from *ScheduledControllers* are the topLevel controllers of each active window, plus an additional controller which manages the main system *yellowButtonMenu* available on the grey screen background. Since each view is associated with a unique controller, the view/subView tree induces a parallel controller tree within each topView. Further branches from each topLevel controller follow this induced tree. Control passes from controller to controller along the branches of this tree.

In simple terms the control flow requires the cooperative action of well bred controllers each of which politely refuses control unless the cursor is in its view. And upon accepting control the well bred controller attempts to defer to some subView's controller. The top level *ControlManager* asks each of the controllers of the active topViews if it wants control. Only the one whose view contains the cursor responds affirmatively and is given control. It, in turn, queries the controllers of its subViews. Again the one that contains the cursor accepts control. This process finds the innermost nested view containing the cursor and, in general, that view's controller retains control as long as the cursor remains in its view. (A more detailed exposition of this control flow appears in Appendix A.) In this scheme, control management involves the cooperation of **all** the active views and controllers in an intricately coordinated minuet. Views are required to poll the controllers of their subViews. Controllers ask their views if they contain the cursor. For that reason, it is unusual -- and risky -- to make modifications to your views or controllers that involve nonstandard flow of control. Keep this firmly in mind when you first attempt to install a new application controller because inadvertent disruption of the flow of control will crash the system. The prudent programmer does a snapshot before making the attempt!

The vital role played by controllers implies that you cannot have a model-view pair without a controller. If that were allowed, the flow of control would disappear in the gap left by the missing controller. Yet there are some cases where you might want a set of subViews to be controlled collectively from the containing controller, rather than from the individual controllers of the subViews. A special controller for such subViews is provided by the class *NoController* which is specifically constructed to refuse control.

The dance of the scroll bars among the browser subviews as the cursor moves between them is the most visible consequence of the flow of control. As the cursor crosses boundaries of subViews within the browser the scroll bar of the just exited subView disappears and the scroll bar for the entered subView appears. This is accomplished by the *controlInitialize* and *controlTerminate* methods of those subViews with controllers that inherit from *ScrollController*. When the cursor exits a given subView, *viewHasCursor* returns false, and the controller executes its *controlTerminate* method which redisplays the area previously covered by the scroll bar. Then the appropriate portion of the view/control tree is traversed to find the controller which should now have control. If this view should have a scroll bar, the *controlInitialize* method of the new controller saves the display area that is to be covered by the scroll

bar, then displays the scroll bar.

Entering and leaving the flow of control

Remember that this minuet of manners is a constantly ongoing one. How then does your newly created MVC triad step into the process, and how does it retire when it is done?

First, the controller of your topViews is the one responsible for entering this process. It then passes control to its subView controllers (which in reality do most of the work in a typical application). The top level controllers must all be descendants of the class *StandardSystemController* which is designed to be the controller of a top level view. The *open* message to a standardSystemController causes your new MVC to become a top level branch of the control tree. The *open* message should be the last message in the method which creates the new MVC because control does not return from this message. Code appearing after the *controller open* message will not be executed. The *controlTerminate* method of a *StandardSystemController* takes responsibility for unscheduling when the window is closed.

The MouseMenuController

Most applications use the mouse for pointing and menu options. Most controllers are therefore installed somewhere in the class hierarchy under *MouseMenuController* which provides the basic instance variables and methods. The relevant instance variables are *{red, yellow, blue}ButtonMenu* and *{red, yellow, blue}ButtonMessages* in which you will install your menus and their associated messages. The important methods are *redButtonActivity*, *yellowButtonActivity*, and *blueButtonActivity*. The *Controller controlLoop* method, as its name implies, is the main control loop. Each time through this loop, it sends *self controlActivity*. This method is reimplemented in MouseMenuController to check each of the mouse buttons and, for instance, send *self redButtonActivity* if the red button is pressed and its view has the cursor. The xxxButtonActivity checks for a non nil xxxButtonMenu, and if found, sends the message: *self menuMessageReceiver perform: (redButtonMessages at: index)*. Note: *menuMessageReciever* normally returns self -- i.e., the controller -- so that menu message protocol normally resides in the controller.

All top level controllers are instances of *StandardSystemController* or its subclasses. The *StandardSystemController* is a subclass of *MouseMenuController*, which is specialized for being at the top level of a window's controller hierarchy. It manages the familiar blueButtonMenu -- the frame, close, collapse, . . . behavior of windows -- which apply to the topView. Your subview controllers should **not** be subclasses of *StandardSystemController*. Rather, they should descend separately from *MouseMenuController*. Yet the blueButton menu functions should still be handled by the topLevel controller. To ensure this your controller can reroute a blueButton press to the top level by the following blueButtonActivity method:

```
blueButtonActivity
view topView controller blueButtonActivity.
```

This is most transparent in that a person browsing your controller code can immediately see that the blueButton is being handled by the topView controller. A more subtle approach is for the subView controller to refuse control if the blueButton is down. This is done in your *isControlActive* method:

```
isControlActive
^super isControlActive & sensor blueButtonPressed not.
```

In the typical case, your controller will have its own *yellowButtonMenu*, and perhaps use the red button for some sort of pointing or item selection function. You will usually make your menus something like the following:

```
PopUpMenu labels:
'foo baz
over there
file out
new gadget'
          lines: #(1 3).
```

This provides a menu with the given options, and with lines after the first and third items. You then install a parallel list of messages, e.g., *#(fooBaz overThere fileOut newGadget)*. The menu and the messages list must end up in the yellowButtonMenu and yellowButtonMessages instance variables. This can be done on the fly if desired, but the more typical approach is to build the menu and messages in a class initialize method and install them in the instance initialize method. For an example, look at the *ChangeListController class initialize* method. Finally, you need to install the methods to implement the menu messages. They conventionally reside in the *menu messages* protocol of your controller. If you wish to use redButton presses for control activity other than menu selection, redefine the *redButtonActivity* of your controller to implement the desired activity. In your new redButtonActivity method you can check for other conditions such as *Sensor leftShiftDown* to provide more flexibility. You can see examples of nonmenu *redButtonActivity* methods in the *FormEditor*, *ListController*, and *SelectionInListController*.

Menus, despite their use of the display screen, are not handled by MVC triads. They manage their own screen display and control, including the saving and replacing of the screen contents in the region covered by the menu. In addition to the PopUpMenu in the above example, you should examine ActionMenu which is especially useful for pluggable views.

ParagraphEditor

All controllers that accept keyboard text input are under the *ParagraphEditor* in the *Controller* hierarchy. *ParagraphEditor* predates the full development of the MVC paradigm. It handles both text input and text display functions, hence it is in some ways a cross between a view and a controller. The views which use it or its subcasses for a controller reverse the usual roles for display; they implement the display of text by sending *controller display.* The multiplicity of roles played by the *ParagraphEditor* make it a complex object. It manages the special keys (for example, the Ctrl T mapping to *ifTrue:*). It also manages the selection of text, the selection of font and point size, and the formatting of the text (that is, proportional character spacing and line breaks tailored to the width of the view). Because it is at the top of the hierarchy of text handling controllers, you have easy access to all of it's power. But the complexity of the text processing classes carries a substantial overhead penalty. This overhead is most visible in the annoying delay between the typing of a character and its display on the screen. The three classes that do most of the text prossesing work are *ParagraphEditor*, *Paragraph*, and *CharacterScanner*. Some Smalltalk-80 programmers have created parallels to each of these classes that dispense with most of the time consuming features. They have achieved text processing speeds for specialized applications that are several times faster than that provided by the standard classes.

ScreenController

The gray screen background and the yellowButtonMenu available on that background are not special exceptions to the MVC paradigm; they too are managed by an MVC triad. The model is an *InfiniteForm* (colored gray), the view is an instance of *FormView*, and the controller is the single instance of class

ScreenController. To add an item to the main screen menu (perhaps a printScreen option), you edit the *ScreenController* class method *initialize* (in the class initialization protocol) and add your method in the menu messages protocol. Don't forget to execute the comment at the bottom of the class initialize method to install your new menu. You might also wish to look at the other methods in the menu messages protocol to see how browsers, file lists, or workspaces are opened.

The MVC Inspector

Because the MVC triad is so important, Smalltalk-80 provides a specialized inspector -- an MVC inspector -- to examine all three objects at once. You will likely make considerable use of this inspector when you begin to build your own applications both as a tool for seeing how other MVC triads work and as an invaluable debugging aid to see why your own doesn't work. You open an MVC inspector by sending the message *inspect* to any View. The easiest way to make use of MVC inspectors is to fileIn the BLUEINSP.ST goodie provided with the Smalltalk-AT distribution. [Note: not all distributions of Smalltalk-80 include this goodie.] It installs a blue button menu item "inspect" for all topViews. This menu item opens an MVC inspector on the view, its model and its controller. Once you understand a bit about the MVC triad, the MVC inspector will let you poke about in the innards of any window which catches your fancy. In a complex view such as a browser, with many subviews, you can follow the chain of subViews down, opening further MVC inspectors on selected subViews.

As an exercise, open an MVC inspector on the System Transcript window (first be sure a SystemTranscript is open on your screen). Begin by opening an inspector on the instances of *DependentsCollection* by executing *DependentsCollection allInstances inspect.* Locate the item with two dependents: "a *StandardSystemView*" and "*aTextCollectorView*." Select the *TextCollectorView* and choose inspect. Now you will have an MVC inspector on the *TextCollector*, its view, and its controller. For instance, in the top model subview, select the *contents* variable: in the right window, you will see the same text as you see in your Transcript window. In the bottom controller section look at and inspect the button menus and messages. They will be the familiar ones of the transcript. In the middle view section, select superView: it will be a *StandardSystemView*. Select it and choose inspect. You will then have another MVC inspector on the topView of the transcript window. These two MVC inspectors together give access to the entire structure of the transcript application. Note, both MVC triads share the same model -- the *TextCollector* .

[Note: In versions of Smalltalk-80 prior to the inclusion of the *Model* class, you will need to start differently: begin by opening an inspector on the *DependentsFields* dictionary (the dictionary in which all object dependencies are maintained) by executing *(Object classPool at: #DependentsFields) inspect.* This gives you access to an inspector on the *IdentityDictionary*. Locate the item with the key "aTextCollector," which is the model of a transcript. Select it and choose the inspect option in the yellowButtonMenu. This opens an inspector on the *OrderedCollection* of objects registered as dependents of the *TextCollector.* In this new inspector, one of the items will be a *TextCollectorView*. Select the *TextCollectorView* and choose inspect.]

Appendix A:

Details on the flow of control

Above all of the controllers of topViews is *ScheduledControllers* -- the instance of *ControlManager* attached to the current project. This controlManager determines which of the topView's controllers should be active. It uses the method *searchForActiveController*, which in turn sends *isControlWanted* to

all scheduledControllers. When it finds the appropriate top level controller it sends that controller the message *startUp*, which is inherited from class Controller. It sends the following three messages: *self controllInitialize, self controlLoop, self controlTerminate*. The *controllLoop* method handles the flow of control by:

```
        [self is ControlActive] whileTrue: [Processor yield.
self controlActivity]
```

The *self controlActivity* just says: *self controlToNextLevel*. Here the control flow is passed briefly to the view by means of the code:

```
        aView := view subViewWantingControl.
        aView ~~ nil ifTrue: [aView controller startUp]
```

which just asks the view to determine which of its subviews wants control, and if one does, passing control to that view's controller. The *isControlWanted* method simply returns the result of the message *self viewHasCursor* which in turn simply says *view containsPoint: sensor cursorPoint*. Thus, at the bottom of the flow of control process is the question of which view contains the cursor. The one exception is *NoController* which reimplements *isControlWanted* to always return false.

Thanks to Charles Lloyd for HTMLizing this document.

Smalltalk Archive | Research Group | Contact Ian | Ian's Home | Original RTF of this document

If you spot an error, please contact Ian. Last modified: Tue Mar 4 13:58:26 1997